

Module 7 Week B — Stretch: Adversarial QA Probe

***Honors Track.** This stretch is for learners who have completed all required Week B work (Reading, Drill, Lab, Integration), are On Track or Advanced, and are attending consistently. Stretch participation is graded but not required for program completion; it counts toward Honors distinction.*

In this stretch you will design a hypothesis-driven adversarial QA test set (~30 examples) targeting one specific failure mode you identified in your Lab 7B analysis, run the same QA model against the adversarial set, and write a memo on what your adversarial set surfaced and how a production team would address it.

The pedagogical goal is to move from *measuring* failure modes (Lab 7B’s qualitative analysis) to *engineering* them — designing inputs that surface a hypothesized weakness and reasoning about defenses.

Due: Tuesday end of day. Resubmissions are accepted through Saturday of the assignment week.

Prerequisite: Lab 7B base assignment must be substantially complete (QA pipeline working, EM/F1 functions implemented, baseline metrics on the tech-news QA set produced).

What You Will Produce

This stretch extends your Lab 7B repo on a new branch `stretch-tue-adversarial-qa`. New files:

- `adversarial_set.csv` — your authored ~30 adversarial examples, columns: `qid`, `question`, `context`, `gold_answer`, `pattern_tag`.
- `adversarial_eval.py` — runs the QA pipeline against `adversarial_set.csv` and computes per-pattern metrics. Reuses `evaluate_qa` from your `lab.py`.
- `adversarial_predictions.csv` — output (same schema as Lab 7B’s `qa_predictions.csv`).
- `adversarial_metrics.json` — aggregate EM/F1 plus a per-`pattern_tag` breakdown.
- `adversarial-analysis.md` — 1-page memo (4 required sections).

Setup

The stretch starter files ship inside your Lab 7B template repo at `stretch/tuesday/` — they were already in the repo when you accepted the Lab 7B GitHub Classroom assignment, so no separate download is needed.

- From your Lab 7B repo:

```
cd <your-lab-7b-repo>
git checkout main && git pull
git checkout -b stretch-tue-adversarial-qa
ls stretch/tuesday/      # adversarial_set.csv, adversarial_eval.py, adversaria...
```
- Implement your work in `stretch/tuesday/`. The autograder workflow only runs on the lab’s main test path; this branch’s work is TA-graded on the Honors rubric.

Step 1: Pick a Failure Mode (and a Hypothesis)

Re-read your Lab 7B `qa-evaluation-report.md`. Pick **one** of the three failure modes you identified, and write it up operationally:

- What input pattern triggers the failure?** (e.g., “the context contains a distractor entity of the same type as the gold entity”)
- What output pattern indicates the failure?** (e.g., “the model returns the distractor instead of the correct entity”)
- Why do you hypothesize the model fails this way?** (e.g., “the QA head learned to lock onto entity type as a strong feature; it doesn’t have enough signal to distinguish ‘the X mentioned in connection with Y’ from ‘the X mentioned anywhere’”)

This is the central engineering act of the stretch. Spending an hour here is well-spent — vague hypotheses produce vague adversarial sets that don’t isolate anything.

Step 2: Author Your Adversarial Set (~30 Examples)

Write 25–40 examples in `adversarial_set.csv`. Constraints:

- Each `gold_answer` must be a **substring of its context** (extractive QA constraint — the autograder verifies this).
- Each row’s `pattern_tag` is your label for which sub-pattern it tests (e.g., `same-type-distractor`, `paraphrased-question`, `multi-sentence-answer`). Use 2–4 distinct tags.
- Include **2–3 control examples** — easy versions of the same pattern that the model *should* get right. The control set is what makes your adversarial set a discriminator (it shows that the pattern, not raw difficulty, drives failure).

Don’t try to make every example a guaranteed failure. The point is to test the hypothesis, not to construct a worst case. A diagnostic test set has both wins and losses — it tells you *when* the model fails, not just *that* it fails.

Step 3: Run the Evaluation

Implement `evaluate_adversarial(qa, df)` in `adversarial_eval.py`:

- Reuse `evaluate_qa` from your `lab.py` (import it).
- After computing aggregate EM/F1, also compute per-`pattern_tag` aggregates.
- Return a dict: `{em, f1, n, per_pattern: {tag: {em, f1, n}}, predictions: [...]}`.

Implement `main()` to read `adversarial_set.csv`, call `evaluate_adversarial`, and write `adversarial_predictions.csv` and `adversarial_metrics.json`.

Run it:

```
python adversarial_eval.py
```

Step 4: Pre-Run Peer Review

Before you run the evaluation against your adversarial set, you review one peer’s `adversarial_set.csv` and design rationale. The peer review is on the *design*, not the results — that’s where the engineering value is.

The 4-question checklist:

- Is the targeted failure mode named **clearly and operationally** (vs. a vague “the model is wrong”)?
- Are the ~30 examples genuinely instances of the targeted pattern (vs. random hard examples)?
- Is each gold answer literally extractable from the context?
- Does the set include 2–3 control examples to confirm the pattern is the discriminator and not raw difficulty?

Submit your review as a comment on the peer’s PR before they run `adversarial_eval.py`. (Pair-assignment is the script’s responsibility; you’ll be paired Tuesday morning.)

Step 5: Write the Memo

`adversarial-analysis.md` has four required sections:

- Hypothesis.** Restate your failure mode operationally (input pattern, output pattern, why-you-hypothesize).
- Set design.** How many examples per `pattern_tag`. Why these tags. What your controls test.
- Results.** Aggregate EM/F1 vs. your Lab 7B baseline (curated tech-news QA). Per-`pattern_tag` breakdown. Cite specific (qid, question, predicted, gold) tuples that illustrate each pattern.
- Production defense.** **One** engineering action that would address the vulnerability you measured. Specific examples: confidence-threshold filter (route below-threshold queries to humans), retraining with adversarial data, replacing the QA model with a calibrated one, restricting the QA model to only trusted contexts that don’t contain distractors. Don’t list five generic defenses — pick one and reason about it concretely.

The memo should fit on one page. Don’t pad. The TA rubric rewards specificity; it penalizes generic ML-safety language.

Submitting

```
git add adversarial_set.csv adversarial_eval.py adversarial_predictions.csv adver
git commit -m "Complete stretch tue: adversarial QA probe"
git push -u origin stretch-tue-adversarial-qa
```

Open a PR from your stretch branch into `main`. PR description: 1-line statement of your targeted failure mode + the EM gap (adversarial vs. tech-news baseline) + 1-line summary of your production defense.

Paste your PR URL into TalentLMS → Module 7 → Stretch Tue (Honors) to submit this assignment.

CI Autograder

A separate stretch autograder workflow runs on your `stretch-tue-*` branch and on PRs that touch `stretch/tuesday/**` — it’s a **structural guardrail** (set schema, gold-extractability, output files exist). Make sure your CI is green before submitting; if it’s red, the autograder’s failure messages tell you what’s structurally wrong with the deliverable.